Ejercicio 6 - Polimorfismo vía Interfaces

CC2008 - Programación Orientada a Objetos, Semestre II, 2026

Parte 1: Análisis y diseño

Análisis del problema

La plataforma de comercio electrónico necesita conectarse con varias APIs externas: pasarelas de pago (tarjetas, transferencias y billeteras digitales), proveedores de logística y certificadores de factura electrónica (FEL). Cada API hace cosas distintas. Unas solo consultan, como rastrear un paquete. Otras cobran y devuelven dinero. Otras generan documentos legales, como el XML de una factura o una guía de envío.

La empresa también quiere poder agregar APIs nuevas en el futuro sin cambiar el código principal. Además, si el sistema pide un reintegro a una API que no maneja dinero (por ejemplo, una de logística), la petición debe ignorarse sin errores y sin detener la compra.

Requisitos funcionales

- Al iniciar, cargar al menos 10 integraciones (el programa carga 12).

- Listar todas las integraciones con su estado de conexión.

- Buscar una integración por su identificador y mostrar sus parámetros de configuración.

- Ordenar el catálogo por latencia promedio en milisegundos.

- Ignorar sin errores las operaciones que una API no soporta.

- Poder agregar una integración nueva sin cambiar la lógica central.

Diseño de clases e interfaces

La clase abstracta IntegracionAPI guarda los datos que todas las APIs tienen: identificador, nombre, proveedor, URL base, estado de conexión, latencia y un historial. Esta es la clase que se guarda en la lista. De ella salen estas clases hijas:

- PasarelaPago (abstracta), con tres hijas: PasarelaTarjeta, TransferenciaBancaria y BilleteraDigital.

- ProveedorLogistico.

- CertificadorFEL.

Los contratos se hicieron con interfaces:

- Transaccional: procesarCobro y procesarReintegro.

- Documentable: generarDocumento.

- Auditable: obtenerHistorial.

- Consultable: consultarEstado.

- Comparable: para ordenar por latencia.

Se agregó la interfaz Consultable porque el enunciado habla de APIs que solo consultan estados, y esas no encajaban en Transaccional, Documentable ni Auditable.

Qué implementa cada clase:

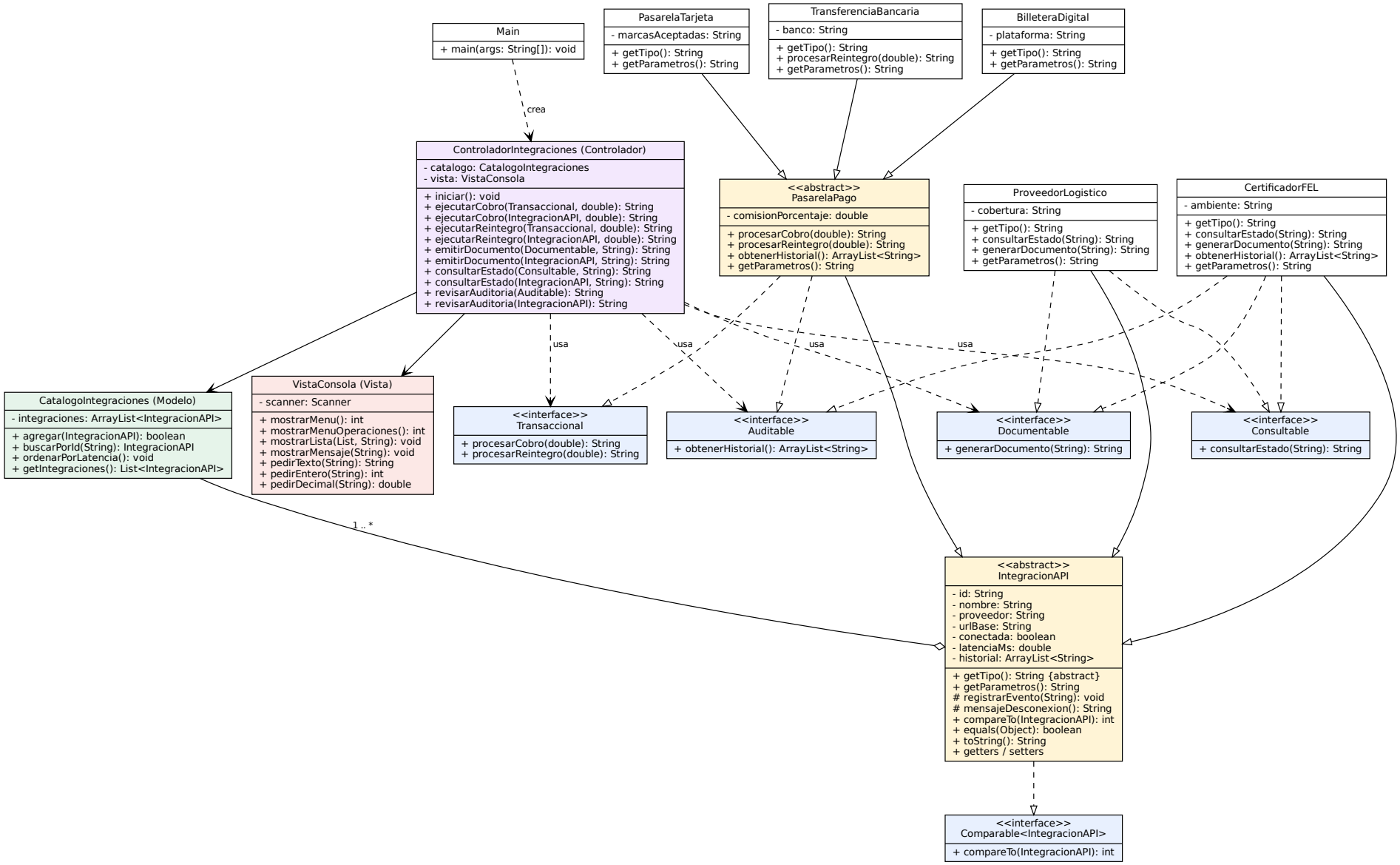
- PasarelaPago y sus hijas: Transaccional y Auditable.

- ProveedorLogistico: Consultable y Documentable. No es Transaccional, por eso no puede hacer reintegros.

- CertificadorFEL: Consultable, Documentable y Auditable.

- IntegracionAPI (todas): Comparable.

Diagrama UML



Una sola lista polimórfica

La clase CatalogoIntegraciones tiene un solo ArrayList de IntegracionAPI. Ahí se guardan todas las integraciones, sin importar su tipo. No hay listas separadas para pagos, logística o FEL. Cuando se recorre la lista, cada objeto responde con su propia versión de getTipo, getParametros y toString. Eso es polimorfismo.

Uso de interfaces y cómo se ignora lo que una API no soporta

El controlador tiene métodos que reciben interfaces como parámetro, por ejemplo ejecutarReintegro(Transaccional t, double monto). Cuando el menú trabaja con una IntegracionAPI, el controlador revisa con instanceof si esa API cumple el contrato. Si lo cumple, llama al método. Si no lo cumple, devuelve un mensaje como "no maneja reintegros, petición ignorada". No se lanza ninguna excepción y el programa sigue normal.

Ejemplo: un reintegro a Cargo Expreso (logística) se ignora. Un reintegro a ACH Banco Industrial se realiza.

Overloading

Cada operación del controlador tiene dos versiones con el mismo nombre: una que recibe la interfaz y otra que recibe IntegracionAPI.

- ejecutarCobro(Transaccional, double) y ejecutarCobro(IntegracionAPI, double)

- ejecutarReintegro(Transaccional, double) y ejecutarReintegro(IntegracionAPI, double)

- emitirDocumento(Documentable, String) y emitirDocumento(IntegracionAPI, String)

- consultarEstado(Consultable, String) y consultarEstado(IntegracionAPI, String)

- revisarAuditoria(Auditable) y revisarAuditoria(IntegracionAPI)

Java escoge la versión según el tipo con el que se llama al método. La versión de IntegracionAPI es la que usa el menú. Esa versión revisa el contrato y, si corresponde, llama a la versión de la interfaz.

Interfaz Comparable

IntegracionAPI implementa Comparable de IntegracionAPI. El método compareTo compara la latencia con Double.compare, de menor a mayor. Así, Collections.sort ordena directamente la lista polimórfica. Las APIs más lentas quedan al final, lo que ayuda a encontrar los cuellos de botella.

Aplicación de MVC

- Modelo: IntegracionAPI, sus hijas, las interfaces y CatalogoIntegraciones. No imprimen nada, solo devuelven datos o texto.

- Vista: VistaConsola. Muestra menús y resultados y lee lo que escribe el usuario. No tiene lógica del negocio.

- Controlador: ControladorIntegraciones. Hace la carga inicial, recibe las opciones del menú, llama al modelo y manda los resultados a la vista.

- Main: crea el modelo, la vista y el controlador y empieza el programa.

Principios de diseño aplicados: cada clase tiene una sola tarea (separación de responsabilidades); el controlador depende de interfaces y no de clases concretas; y el sistema está abierto a crecer sin modificar lo que ya existe.

Buenas prácticas

- Los atributos son privados y se usan getters y setters.

- toString está sobrescrito en IntegracionAPI para mostrar cada integración en una línea.

- equals y hashCode están sobrescritos y comparan por identificador. Así el catálogo no permite identificadores repetidos.

- Se usa @Override en los métodos sobrescritos y cada clase tiene su comentario de encabezado.

Cómo se agrega una integración nueva

Cuando la empresa contrate a la startup de entregas de última milla, solo hay que crear el objeto, por ejemplo new ProveedorLogistico(...), y agregarlo con catalogo.agregar. Funciona con los mismos contratos y no se cambia nada del controlador ni de la lógica de las operaciones. En el prototipo, Forza Delivery (LOG-03) es un ejemplo de este caso. Si fuera un tipo de API totalmente nuevo, se crea una clase hija de IntegracionAPI que implemente los contratos que necesite.

Datos de la carga inicial

PAY-01 Visanet Guatemala - Pasarela de tarjeta - 180 ms

PAY-02 Credomatic - Pasarela de tarjeta - 220 ms

PAY-03 ACH Banco Industrial - Transferencia bancaria - 350 ms

PAY-04 Transferencia Banrural - Transferencia bancaria - 410 ms (desconectada)

PAY-05 Tigo Money - Billetera digital - 140 ms

PAY-06 PayPal - Billetera digital - 260 ms

LOG-01 Cargo Expreso - Proveedor logístico - 300 ms

LOG-02 Guatex - Proveedor logístico - 280 ms

LOG-03 Forza Delivery - Proveedor logístico - 120 ms

FEL-01 Infile - Certificador FEL - 450 ms

FEL-02 Megaprint - Certificador FEL - 520 ms

FEL-03 Digifact - Certificador FEL - 600 ms (desconectada)

Cambios durante la implementación

El diseño no cambió durante la implementación. El programa sigue el diagrama UML tal como está.

Cómo compilar y ejecutar

Con todos los archivos .java en la misma carpeta: javac *.java y luego java Main. El menú tiene estas opciones: listar integraciones, buscar por ID, ordenar por latencia, probar una operación (cobro, reintegro, documento, consulta o historial) y salir.